



**UNIVERSITÀ
DEGLI STUDI
DI UDINE**
HIC SUNT FUTURA

DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE

TESI DI LAUREA IN
INFORMATICA

RAM-USB

Progettazione e implementazione di un server di backup multi utente Zero-Knowledge e Zero-Trust

CANDIDATO

Francesco Verrengia

RELATORE

Prof. Ivan Scagnetto

Anno accademico 2025-2026

CONTATTI DELL'ISTITUTO

Dipartimento di Scienze Matematiche, Informatiche e Fisiche

Università degli Studi di Udine

Via delle Scienze, 206

33100 Udine — Italia

+39 0432 558400

<https://www.dmif.uniud.it/>

Sommario

Nunc ac dignissim ipsum, quis pulvinar elit.

Indice

1	Introduzione	1
1.1	Contesto e motivazione	1
1.2	Problema affrontato	1
1.3	Obiettivi e perimetro	1
1.4	Metodologia	1
1.5	Struttura della tesi	1
2	Stato dell'arte e contesto tecnologico	3
2.1	Principi di sicurezza di riferimento	3
2.2	Soluzioni di backup esistenti	3
2.3	Paralleli in altri domini a conoscenza zero	3
2.4	Reti mesh e zero-trust networking	3
2.5	Posizionamento di RAM-USB	3
3	Requisiti: metodologia e casi d'uso	5
3.1	Metodologia dei requisiti	5
3.2	Tracciabilità dei requisiti	5
3.3	Requisiti utente e casi d'uso	6
4	Architettura generale e trust zones	9
4.1	Overview architetturale	9
4.1.1	Entry-Hub	10
4.1.2	Security-Switch	10
4.1.3	Database-Vault	10
4.1.4	Storage-Service	11
4.1.5	Network-Manager	12
4.1.6	Certificate-Authority	12
4.1.7	Headscale	12
4.1.8	Mosquitto	13
4.1.9	Metrics-Collector	13
4.1.10	Grafana	13
4.2	Modello di comunicazione inter-servizio	13
4.3	Trust zones e confini di sicurezza	14
4.4	PKI e gestione dei certificati	14
5	Autenticazione e gestione delle identità	17
5.1	Ciclo di vita della richiesta	17
5.2	Cifratura e hashing delle credenziali	18
5.2.1	Robustezza della password	20
5.3	Registrazione	21
5.4	Login	23
5.5	Orchestrazione post-autenticazione	25

6	Storage e backup dei file	27
6.1	Modello di accesso SFTP-only	27
6.2	Isolamento e chroot dinamico	27
6.3	Backup e restore	27
7	Rete e coordinamento della mesh	29
7.1	Politiche di rete e confini di reachability	29
7.2	Ciclo di vita dei grant di accesso	29
7.3	Il paradosso del coordinatore: Headscale come servizio a sé	29
7.4	CG-NAT e la necessità di un endpoint pubblico dedicato	29
8	Osservabilità e metriche	31
8.1	Pubblicazione delle metriche	31
8.2	Ingestione e controllo di coerenza	31
8.3	Persistenza delle serie temporali	31
8.4	Visualizzazione delle metriche	31
9	Testing e validazione	33
9.1	Il modello V e i quattro livelli di test	33
9.2	Strategia di integrazione bottom-up	33
9.3	TDD: criteri di ingresso e uscita	33
9.4	Copertura e lacune del piano di test	33
9.5	Risultati: verifiche completate	33
10	Deployment	35
10.1	Containerizzazione e supervisione s6-overlay	35
10.2	Perché non tutti i container sono distroless	35
10.3	Docker locale vs Proxmox VE: due livelli di isolamento	35
10.4	Topologia distribuita	35
10.5	Alta disponibilità e resilienza: stato attuale e limiti	35
11	Analisi critica delle proprietà di sicurezza	37
11.1	Scenari di attacco e mitigazioni	37
11.2	Costo di un attacco alle password	37
11.3	Isolamento dei fallimenti	38
11.4	Costo delle scelte architetturali	38
12	Conclusioni	39
12.1	Risultati raggiunti	39
12.2	Limiti noti	39
12.3	Sviluppi futuri	39
A	Elenco completo dei requisiti	41
A.1	Requisiti di sistema funzionali	41
A.1.1	Client	41
A.1.2	Entry-Hub	42
A.1.3	Security-Switch	43
A.1.4	Database-Vault	44
A.1.5	Storage-Service	45
A.1.6	Network-Manager	46
A.1.7	Certificate-Authority	47
A.1.8	Sistema di monitoraggio	47
A.1.9	Infrastruttura di rete e PKI	48
A.2	Requisiti non funzionali	48

A.2.1	Di prodotto	48
A.2.2	Organizzativi	49
A.2.3	Esterni	49
A.3	Requisiti di dominio	49

1

Introduzione

1.1 Contesto e motivazione

1.2 Problema affrontato

1.3 Obiettivi e perimetro

1.4 Metodologia

1.5 Struttura della tesi

Il Capitolo 4 presenta i componenti che partecipano alla registrazione e al login. Il Capitolo 5 segue le credenziali lungo quel percorso, dal confine pubblico fino al record salvato, e descrive le regole che ogni strato applica.

2

Stato dell'arte e contesto tecnologico

- 2.1 Principi di sicurezza di riferimento
- 2.2 Soluzioni di backup esistenti
- 2.3 Paralleli in altri domini a conoscenza zero
- 2.4 Reti mesh e zero-trust networking
- 2.5 Posizionamento di RAM-USB

Requisiti: metodologia e casi d'uso

3.1 Metodologia dei requisiti

I requisiti di RAM-USB sono raccolti in un documento unico chiamato Software Requirements Specification (SRS) [1], che costituisce l'unica fonte di verità su cosa il sistema debba fare. La scrittura di tale documento segue un modello a spirale dell'ingegneria del software [2], guidato dalla progressiva risoluzione del rischio, inteso come le fonti di incertezza più significative del progetto. Il livello di dettaglio della SRS aumenta di conseguenza ad ogni iterazione. Gli obiettivi e i requisiti di alto livello erano chiari fin dalle fasi iniziali. La conoscenza degli strumenti e delle tecnologie di terze parti, però, non bastava a prevederne con certezza il comportamento a basso livello. La scelta del modello a spirale nasce da questo squilibrio. Molti vincoli concreti sono infatti emersi solo nel momento dell'implementazione, quando l'interazione diretta con questi strumenti ha rivelato dettagli che una fase di analisi puramente teorica, non è stata in grado di anticipare. Un caso concreto di questa dinamica è il requisito NM-F-12, rivisitato in tre round successivi:

- nella versione 1.0 della SRS [3] assumeva che Network-Manager (il componente responsabile della rete privata del sistema) dovesse costruire da zero un proprio meccanismo per autorizzare l'accesso di nuovi dispositivi;

- nella versione 1.4 [4] è emerso che Headscale, lo strumento di coordinamento della rete impiegato dal sistema, offriva già questa funzionalità tramite la propria CLI, eliminando la necessità di scriverla;

- nella versione 1.7 [5] è emerso un limite più profondo dello stesso Headscale (non può coordinare una rete di cui fa parte) approfondito nella sezione 7.3, richiedendo una riformulazione più ampia del requisito.

3.2 Tracciabilità dei requisiti

Un aspetto metodologico distintivo del progetto è la tracciabilità, intesa come vincolo tecnico verificabile in tre direzioni indipendenti. Questo approccio privilegia la semplicità di un audit di sicurezza a scapito della complessità della gestione dell'implementazione.

- **Dal requisito alla sua implementazione:** ogni requisito della SRS che ha già un'implementazione è collegato a un permalink GitHub che punta a un commit specifico e ad un intervallo di righe preciso.
- **Dal test al requisito e viceversa:** ogni funzione di test porta, come commento di documentazione nella riga immediatamente precedente, l'identificativo del requisito che verifica. La relazione non è necessariamente 1-a-1 visto che un requisito può avere più test che lo verificano. Inoltre, la tracciabilità vale in entrambe le direzioni: si può infatti passare dal requisito ai test, cercando l'ID nella codebase; e dal test al requisito, leggendo il commento sovrastante la funzione.
- **Dal commit al requisito:** ogni commit del repository segue una grammatica fissa, descritta nel `CONTRIBUTING.md` [6] e qui sintetizzata come tipo, area, descrizione del cambiamento, seguiti dall'elenco degli identificativi dei requisiti che copre. Come per il collegamento test-requisito, anche questo vale in entrambe le direzioni: dal requisito, se ne cerca l'ID nella storia del repository; nella direzione opposta, l'elenco finale della descrizione di ogni commit indica subito quali requisiti sono coperti.

3.3 Requisiti utente e casi d'uso

I requisiti utente (RU) esprimono, dal punto di vista di chi usa il sistema, cosa RAM-USB deve garantire. Sono gli unici requisiti riportati integralmente nel corpo della tesi; i requisiti di sistema che li realizzano (funzionali, non funzionali e di dominio) sono troppo numerosi per un elenco completo, e sono raccolti in Appendice A.

RU-01 Registrarsi al servizio fornendo la quantità minima di dati necessaria.

RU-02 Potersi autenticare in sessioni successive alla registrazione.

RU-03 Poter caricare backup cifrati verso il servizio da qualsiasi luogo.

RU-04 Poter accedere al proprio backup solo dopo essersi autenticato.

RU-05 Avere la garanzia che nessuno, amministratori inclusi, possa leggere i propri backup.

RU-06 Avere la garanzia che nessuno, amministratori inclusi, possa leggere le proprie credenziali di accesso.

RU-07 Poter recuperare i propri file in qualsiasi momento.

RU-08 Avere la certezza che i propri dati siano isolati da quelli degli altri utenti.

RU-09 (*Fuori perimetro*) Che nessuno, amministratori inclusi, possa modificare o cancellare i propri backup.

RU-10 (Amministratore) Poter osservare la salute e le prestazioni del sistema senza compromettere i dati degli utenti.

I requisiti utente si traducono nei seguenti casi d'uso:

UC-01 (Registrazione) Un nuovo utente fornisce email, password e chiave pubblica SSH, e ottiene un account attivo.

UC-02 (Autenticazione) Un utente registrato fornisce email e password, e ottiene un accesso temporaneo al proprio spazio di backup.

UC-03 (Backup di un file) Un utente autenticato carica un file, già cifrato sul proprio dispositivo, nel proprio spazio isolato.

UC-04 (Restore di un file) Un utente autenticato recupera un proprio file precedentemente caricato, decifrandolo localmente.

UC-05 (Consultazione delle metriche operative) Un amministratore consulta lo stato di salute e le prestazioni del sistema, senza accedere ai dati dei singoli utenti.

La Figura 3.1 mostra questi cinque casi d'uso.

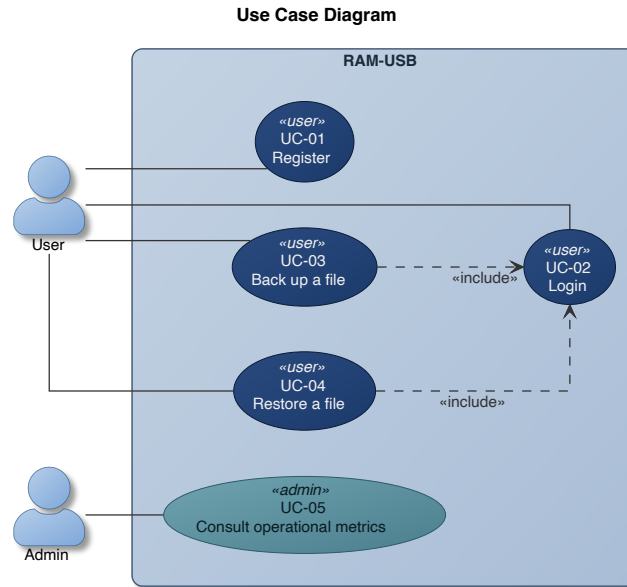


Figura 3.1: Diagramma dei casi d'uso principali di RAM-USB.

I capitoli successivi descrivono come l'architettura del sistema realizzi questi casi d'uso: il capitolo 4 introduce la struttura generale dei componenti e i confini di sicurezza tra di essi, mentre i capitoli 5-8 approfondiscono ciascun dominio funzionale nel dettaglio.

Architettura generale e trust zones

4.1 Overview architetturale

RAM-USB è un'architettura a microservizi n-tier in cui ogni componente ha una responsabilità specifica. Questa separazione riduce il rischio che la compromissione di un componente comprometta l'intero sistema.

RAM-USB ha dieci componenti interni, organizzati in tre gruppi funzionali:

- **Accesso e registrazione:** Entry-Hub, Security-Switch, Database-Vault, Network-Manager, Certificate-Authority, Headscale.
- **Backup e restore:** Storage-Service.
- **Osservabilità:** Mosquitto, Metrics-Collector, Grafana.

La Figura 4.1 mostra questi dieci componenti e il protocollo con cui interagiscono.

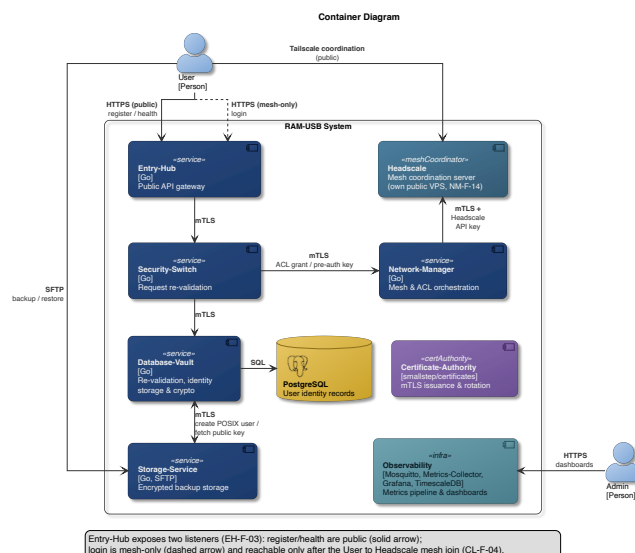


Figura 4.1: Diagramma dei container di RAM-USB.

4.1.1 Entry-Hub

Entry-Hub è il gateway pubblico del sistema, ovvero l'unico esposto a chiunque su internet. Per questo il sistema non gli concede fiducia implicita. Il suo processo gira quindi in un container distroless [7], seguendo il principio di riduzione dell'attack surface raccomandato da NIST SP 800-190 [8]. Senza shell, infatti, la maggior parte delle vulnerabilità critiche note nei container non è sfruttabile [9].

Entry-Hub espone tre endpoint HTTPS: `GET /api/health` e `POST /api/users` sono raggiungibili da chiunque su internet, con certificato firmato dalla CA pubblica Let's Encrypt [10]. `POST /api/login` usa lo stesso certificato, ma ha un listener separato, raggiungibile solo da utenti con accesso alla rete privata, descritta in dettaglio nel Capitolo 7.

Quando riceve una richiesta, Entry-Hub valida il corpo JSON in due fasi: in decodifica, impone un limite di dimensione e rifiuta qualunque campo non atteso nel payload; poi valida i singoli campi email, password e, solo in fase di registrazione, la chiave SSH [11]. Se una di queste verifiche fallisce, risponde HTTP 400 senza specificare quale sia il problema e senza inoltrare la richiesta agli strati più interni del sistema. Se la validazione riesce, inoltra la richiesta a Security-Switch via mTLS verificando che il certificato ricevuto appartenga davvero a un Security-Switch. Attende la risposta del Security-Switch e la rilancia al client.

4.1.2 Security-Switch

Security-Switch esiste per evitare che le responsabilità di contattare Database-Vault e Network-Manager siano in mano al punto di ingresso del sistema: riceve le richieste in ingresso solo da Entry-Hub, autenticato via mTLS, le rivalida in modo indipendente e contatta i servizi interni.

Lo scopo della rivalidazione è l'applicazione del principio di defence in depth [12]. La logica di validazione, gestione degli errori e logging viene riutilizzata da tutti i servizi principali e vive in librerie interne condivise [13].

Se la validazione riesce, inoltra la richiesta a Database-Vault via mTLS.

Nel caso di una richiesta di registrazione, chiede al Network-Manager di creare l'identità del nuovo utente sulla rete mesh e la credenziale (pre-auth key) con cui il suo dispositivo potrà unirvisi.

Nel caso di una richiesta di login, chiede al Network-Manager di concedere un accesso temporaneo di 12 ore per quell'utente verso Storage-Service. Solo quando l'utente avrà ottenuto l'accesso allo Storage-Service, potrà contattarlo e gestire i propri backup.

4.1.3 Database-Vault

Database-Vault è il componente che si occupa di memorizzare le credenziali degli utenti in un database PostgreSQL. È l'unico in grado di accedere al database, ma non ha tuttavia accesso ai file degli utenti, i quali si trovano persistiti sullo Storage-Service. Questa separazione limita l'impatto della compromissione di un singolo componente.

Il diagramma Entità-Relazioni è illustrato in Figura 4.2. Visto che il database deve contenere solo i dati degli utenti e non vi è alcuna relazione significativa da salvare, l'ER è piuttosto semplice.

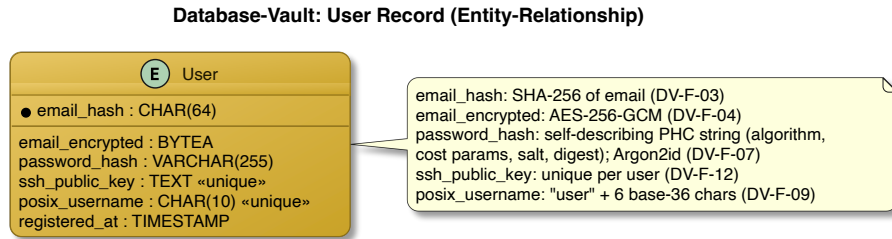


Figura 4.2: Diagramma Entità-Relazioni del record utente in Database-Vault.

L'email è cifrata, recuperabile solo con la master key, mentre la password non è mai recuperabile, poiché viene hashata con Argon2id. Il furto o l'accesso non autorizzato al database non bastano quindi a leggere né l'una né l'altra.

In fase di registrazione di un nuovo utente, in seguito alla rivalidazione dell'input, Database-Vault tenta di salvare i dati in una transazione atomica, rifiutando la richiesta se l'email o la chiave SSH sono già registrate, ma senza specificare quale delle due sia errata. Se il salvataggio riesce, richiede a Storage-Service la creazione dello spazio POSIX dell'utente. In caso di fallimento dello Storage-Service, elimina il record appena creato, altrimenti informa Security-Switch che la registrazione è andata a buon fine.

Al login, ricalcola l'hash della password ricevuta e lo confronta con quello salvato, poi risponde a Security-Switch.

Un'email inesistente e una password sbagliata producono la stessa risposta: in questo modo, chi tenta l'accesso non può usarla per scoprire quali email sono registrate. Il Capitolo 11, tuttavia, spiega come un utente esterno può tentare di capire se un'email è già registrata nel sistema e anche come un amministratore può ottenere le credenziali di accesso di un utente.

4.1.4 Storage-Service

Storage-Service ospita i file di backup degli utenti. Non elabora mai contenuto in chiaro poiché per design, non è in grado di farlo. Riceve infatti soltanto file deduplicati, compressi e cifrati lato client con Restic [14].

Su richiesta di Database-Vault, crea un utente POSIX dedicato, senza shell interattiva né home directory tradizionale. Ogni utente ha infatti uno spazio scrivibile in una sua sottodirectory dentro un chroot. La struttura del file system è illustrata in Figura 4.3.

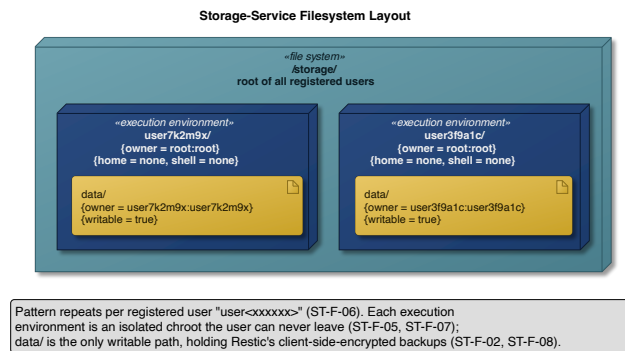


Figura 4.3: Struttura del filesystem di Storage-Service.

L'accesso allo Storage-Service può avvenire solo via SFTP, con la chiave pubblica SSH inviata dall'utente in fase di registrazione. Non sono quindi permessi né l'autenticazione con username e password, né altre forme di accesso SSH al di fuori della SFTP.

Ad ogni connessione, un binario `AuthorizedKeysCommand` dedicato interroga Database-Vault per ottenere la chiave pubblica corrente dell'utente. Qualunque errore in quella chiamata, nega la connessione secondo il principio fail-secure [15].

4.1.5 Network-Manager

Network-Manager è l'amministratore delle regole di rete. Si occupa di definire, modificare ed eliminare le regole che permettono ad un utente della rete privata mesh di raggiungere gli altri membri.

In fase di registrazione, crea un utente Headscale dedicato e la pre-auth key con cui il dispositivo dell'utente si unirà alla rete.

Durante il login, assegna al nodo dell'utente il tag ACL (Access Control List) che abilita l'accesso temporaneo di 12 ore allo Storage-Service. Un processo periodico controlla le scadenze e rimuove il tag dai nodi scaduti.

4.1.6 Certificate-Authority

Certificate-Authority è la CA privata del sistema, basata su `smallstep/certificates` [16].

Emette, rinnova e revoca i certificati mTLS di ogni servizio interno, il che lo rende un componente particolarmente critico del sistema, nonché l'unico raggiungibile da tutti i servizi interni: un malintenzionato che riuscisse a comprometterla, infatti, potrebbe emettere certificati falsificati per qualunque organizzazione.

Ogni servizio ottiene il suo primo certificato contattando la CA sfruttando un token di bootstrap distribuito manualmente da un operatore fuori banda. Una spiegazione dettagliata si trova nella sezione 4.4.

4.1.7 Headscale

Headscale è un'implementazione self-hosted del Tailscale control server [17].

Svolge il ruolo di coordinatore della rete privata mesh: ad ogni tentativo di connessione tra due nodi, fa rispettare le regole di raggiungibilità decise da Network-Manager. Distribuisce inoltre, tramite

MagicDNS, i nomi host con cui i nodi si risolvono reciprocamente, permettendo al client di individuare Storage-Service senza conoscerne l'indirizzo IP.

Valida inoltre la pre-auth key di un nuovo dispositivo al momento dell'ingresso nella mesh.

A differenza di ogni altro componente interno, Headscale non risiede nella rete privata, ma è raggiungibile da chiunque su internet. La sezione 7.3 ne approfondisce le ragioni.

4.1.8 Mosquitto

Mosquitto è il broker MQTT su cui ogni servizio, una volta al minuto, pubblica le proprie metriche operative.

Applica una ACL in cui ogni servizio può solo scrivere sul proprio topic, mai leggerlo, mentre Metrics-Collector può solo leggere l'intero spazio `metrics/*`, senza mai scrivervi.

4.1.9 Metrics-Collector

Metrics-Collector è il componente che riceve, valida e rende persistenti le metriche operative pubblicate da ogni servizio.

Scarta ogni metrica ricevuta il cui campo `service` non corrisponde al topic da cui arriva e persiste le metriche accettate come hypertable in TimescaleDB [18], con ritenzione di trenta giorni e compressione dopo sette giorni.

4.1.10 Grafana

Grafana è la piattaforma di visualizzazione con cui un amministratore consulta lo stato di salute e le prestazioni del sistema [19].

Legge da TimescaleDB direttamente, senza passare per Metrics-Collector. La dashboard di Grafana mostra tempi di risposta, throughput e connessioni attive. Il Capitolo 8 tratta questa pipeline di osservabilità nel dettaglio.

4.2 Modello di comunicazione inter-servizio

Ogni servizio espone la propria API HTTPS sulla libreria standard di Go, `net/http`. Le richieste e le risposte sono corpi JSON ed ogni endpoint è registrato su un router minimale, `http.ServeMux`.

Gli endpoint qui elencati seguono le convenzioni REST [20].

- GET `/api/health`, POST `/api/users` di Entry-Hub,
- GET `/internal/v1/public-key/{posix_username}` di Database-Vault,
- POST `/internal/v1/posix-users` di Storage-Service e
- POST `/internal/v1/mesh-users`, POST `/internal/v1/grants` di Network-Manager

POST `/api/login` è l'unica eccezione, perché un login riuscito restituisce solo la conferma che un'azione è avvenuta ma manca l'identificativo che una vera risorsa `sessions` richiederebbe per essere letta con GET o cancellata con DELETE. Il token ACL di 12 ore che il login concede, infatti, resta interno a Network-Manager e scade autonomamente.

La pubblicazione delle metriche inoltre, è asincrona: ogni servizio, infatti, pubblica i propri messaggi su un topic MQTT dedicato, e non c'è alcun bisogno di attendere una risposta.

Questa struttura ha però un costo non indifferente: ogni registrazione e ogni login attraversano quattro processi distinti, ognuno con il proprio handshake mTLS e la propria rivalidazione. La latenza è quindi ben più alta rispetto ad un sistema monolitico, e non è possibile ridurla saltando uno dei passaggi qualora fosse necessario. Il servizio, inoltre, resta disponibile agli utenti solo finché restano disponibili i sette componenti che partecipano al percorso di una richiesta, mentre la catena di osservabilità può cadere senza che l'utente se ne accorga.

I due costi non sono però dello stesso tipo. La latenza è definitiva, e si accetta insieme all'architettura; la disponibilità è invece un problema di deployment, e il Capitolo 10 discute fino a che punto l'infrastruttura sottostante possa attenuarlo. Il Capitolo 11 riprende invece il bilancio complessivo, mettendo accanto a ogni costo la proprietà che compra.

4.3 Trust zones e confini di sicurezza

La Figura 4.4 mostra le regole di raggiungibilità che Network-Manager applica alla rete mesh.

Le due zone più esterne appartengono all'utente, prima e dopo l'autenticazione. Un utente registrato ma non ancora autenticato, che appartiene quindi alla *Unauthenticated User Zone* vede e può contattare solo Entry-Hub. Dopo il login, lo stesso nodo entra nella *Authenticated User Zone* e ottiene la raggiungibilità temporanea verso lo Storage-Service.

Solo la Certificate-Authority ha un accesso davvero universale: raggiunge, ed è raggiunta da ogni altro componente interno, per permettergli di rinnovare il proprio certificato X.509 senza un intermediario.

La quarta zona, *Public Zone*, contiene soltanto Headscale, il quale è raggiungibile dalla rete pubblica. La sezione 7.3 ne spiega il motivo nel dettaglio. Tuttavia, le chiamate di Network-Manager verso Headscale sono comunque protette dall'autenticazione mTLS o dal token di bootstrap.

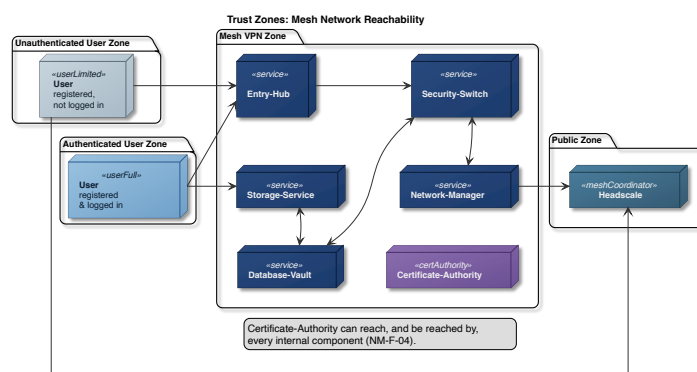


Figura 4.4: Trust zones e raggiungibilità sulla rete mesh.

4.4 PKI e gestione dei certificati

RAM-USB usa due autorità di certificazione, come illustrato in Figura 4.5.

Let's Encrypt, una CA pubblica, firma soltanto il certificato che Entry-Hub presenta sui propri endpoint pubblici. Questo perché il certificato deve essere verificabile da un client esterno che non ha

ancora accesso alla rete interna del sistema. Un certificato firmato dalla Certificate-Authority privata di RAM-USB non sarebbe attendibile.

Ogni altra comunicazione usa invece una Certificate-Authority privata, basata su `smallstep/certificates`.

Ogni certificato privato segue lo stesso schema di bootstrap: ogni servizio riceve, fuori banda e una sola volta, un token di bootstrap monouso, e lo presenta alla Certificate-Authority in cambio del proprio certificato iniziale. I rinnovi successivi presentano il certificato mTLS già posseduto. Il token resta un segreto usa-e-getta e non necessita quindi di essere protetto una volta consumato.

Un certificato X.509 valido dimostra solo che la Certificate-Authority ha approvato quel client, non che sia il client atteso per quella connessione. Per questo ogni servizio verifica anche il campo `organization` del certificato.

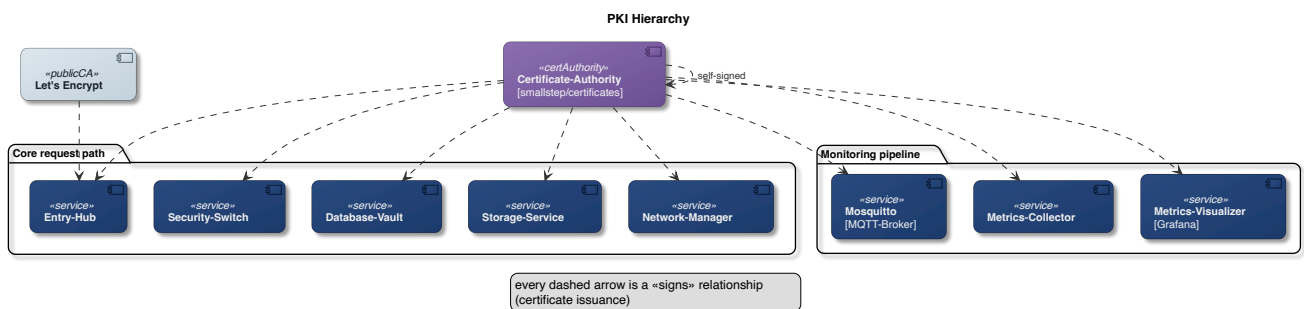


Figura 4.5: Gerarchia delle Certificate Authority e relazioni di firma.

5

Autenticazione e gestione delle identità

5.1 Ciclo di vita della richiesta

Una richiesta di registrazione o di login attraversa tre servizi prima di diventare un record: Entry-Hub, Security-Switch e Database-Vault. Tutti e tre applicano le stesse verifiche sull'input, ognuno indipendentemente dagli altri.

La validazione avviene in due fasi. La prima riguarda il corpo della richiesta: il payload viene letto fino a un limite di 2048 byte e rifiutato se lo supera, senza accumulare in memoria dati controllati da chi ha inviato la richiesta, e il decoder JSON respinge qualunque campo inatteso.

La seconda fase riguarda i singoli campi [11]. L'email deve essere un indirizzo singolo e sintatticamente valido secondo RFC 5322 [21]; la password deve essere lunga tra 8 e 128 caratteri e usare almeno tre categorie tra minuscole, maiuscole, cifre e simboli. In registrazione, inoltre, la chiave pubblica SSH deve essere ben formata.

Se una verifica fallisce, la risposta è HTTP 400 senza indicare quale controllo non sia passato, il log registra il problema senza identificare l'utente, e la richiesta non prosegue verso l'interno. Il messaggio resta generico perché un errore dettagliato aiuterebbe un attaccante più di quanto aiuti un utente in buona fede, che le stesse verifiche le ha già ricevute dal proprio client.

I tre servizi condividono la stessa libreria di validazione, quindi ripetere il controllo non introduce diversità algoritmica, ma garantisce che nessun servizio dia per scontato che il precedente abbia fatto il proprio lavoro. In questo modo, se a Database-Vault dovesse arrivare una richiesta proveniente da un servizio compromesso, la validazione verrebbe fatta comunque.

Gli errori interni vengono infine tradotti in codici HTTP. La Tabella 5.1 li riassume, indicando per ciascuno se a costruirlo sia Entry-Hub, Security-Switch o entrambi.

Tabella 5.1: Codici HTTP restituiti al client e loro significato nel sistema.

Codice	Significato	Emesso da
400	Validazione fallita, senza indicare quale controllo non sia passato	entrambi
401	Credenziali non valide, non specifica se email inesistente o password errata	Security-Switch
403	Rifiuto esplicito di Network-Manager	Security-Switch
409	Email o chiave SSH già registrate, non specifica quale delle due	Security-Switch
500	Errore interno non altrimenti classificato	entrambi
502	Servizio a valle irraggiungibile o con risposta inattesa	entrambi
503	Chiamata a Security-Switch scaduta	Entry-Hub
504	Chiamata a Database-Vault o Network-Manager scaduta	Security-Switch

Entry-Hub costruisce soltanto gli errori che nascono da sé stesso: una validazione fallita e una chiamata a Security-Switch andata male. Tutto il resto lo rilancia al client così come gli arriva dall'interno, senza modificarlo. Per questo motivo, un 401 o un 409 raggiungono l'utente pur non essendo mai prodotti da Entry-Hub.

Tra i codici che i due servizi costruiscono per conto proprio ci sono due differenze: la prima è il 403, che solo Security-Switch può emettere, perché è l'unico dei due a poter ricevere un rifiuto esplicito da un servizio interno. La seconda riguarda le chiamate scadute: Entry-Hub risponde 503, Security-Switch 504.

In tutti i casi il corpo della risposta è sanificato, e il dettaglio dell'errore resta nei log interni.

5.2 Cifratura e hashing delle credenziali

Database-Vault tratta i dati che riceve in tre modi diversi:

- la chiave SSH è pubblica, quindi non necessita di segretezza;
- l'email è insieme una chiave di ricerca e un dato personale da proteggere;
- la password non deve essere recuperabile, nemmeno da chi amministra il database.

L'email viene normalizzata in minuscolo e ridotta al proprio digest SHA-256, che diventa la chiave primaria del record. La normalizzazione garantisce che lo stesso indirizzo scritto con lettere diverse al login produca la stessa chiave della registrazione. Questo hash è deterministico per necessità, perché deve permettere il lookup. Il limite fondamentale di questo approccio, insieme alla sua mitigazione, è discusso nel Capitolo 11.

Accanto al digest che fa da chiave di ricerca, il record conserva una copia cifrata, così che l'indirizzo resti recuperabile. Da una master key e da un salt casuale di 16 byte, generato per quel record, viene derivata con HKDF-SHA256 [22] una chiave dedicata, con cui l'email è cifrata in AES-256-GCM [23] usando un nonce casuale di 12 byte. Salt, nonce e testo cifrato vengono salvati nel record, mentre la

chiave derivata viene azzerata appena il cifrario l'ha consumata. Derivare una chiave per ogni record, invece di cifrare tutto con la master key, riduce la quantità di dati protetti da una singola chiave.

```
func newGCM(masterKey, salt []byte) (cipher.AEAD, error) {
    derivedKey := make([]byte, derivedKeySize)
    kdf := hkdf.New(sha256.New, masterKey, salt, []byte(hkdfInfo))
    if _, err := io.ReadFull(kdf, derivedKey); err != nil {
        return nil, fmt.Errorf("encryption: derive key: %w", err)
    }

    block, err := aes.NewCipher(derivedKey)
    for i := range derivedKey {
        derivedKey[i] = 0
    }
    if err != nil {
        return nil, fmt.Errorf("encryption: new AES cipher: %w", err)
    }

    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, fmt.Errorf("encryption: new GCM: %w", err)
    }

    return gcm, nil
}
```

Codice 5.1: Derivazione della chiave per record in `newGCM`, da `encryption.go` [24].

La password viene invece concatenata a un pepper, letto da una variabile d'ambiente, e passata ad Argon2id [25] insieme a un salt casuale di 16 byte generato sul momento per quel record.

OWASP [26] propone diverse configurazioni equivalenti tra loro, che scambiano memoria e numero di passaggi. RAM-USB parte da quella con più memoria e ne raddoppia i passaggi, pagando quindi a ogni verifica il doppio del lavoro che OWASP considera sufficiente. È un costo che rende impraticabile recuperare le password da un database rubato. Il pepper complica ulteriormente un attacco offline: a differenza del salt, che è salvato nel record e impedisce soltanto che un tentativo valga per più utenti insieme, il pepper nel database non compare: in sua assenza nessun tentativo è calcolabile, e una copia del database non è quindi sufficiente a condurre l'attacco.

Il risultato viene salvato come stringa autodescrittiva in formato PHC, la convenzione di codifica nata dalla Password Hashing Competition. Ogni record porta quindi con sé i parametri con cui è stato creato, e la verifica al login li usa al posto delle costanti correnti del servizio. In questo modo, cambiare i parametri per i nuovi utenti non invalida i record già esistenti.

La Tabella 5.2 elenca i campi di quella stringa, nell'ordine in cui compaiono nel record.

Tabella 5.2: Campi della stringa PHC in cui Database-Vault salva l'hash della password.

Campo	Significato
<code>argon2id</code>	Variante dell'algoritmo
<code>v=19</code>	Versione di Argon2: il byte 0x13 fissato da RFC 9106, stampato in decimale
<code>m=47104</code>	Memoria di lavoro in KiB, cioè 46 MiB
<code>t=2</code>	Numero di passaggi sulla memoria
<code>p=1</code>	Grado di parallelismo
<code>salt</code>	Salt casuale di 16 byte, generato per quel record
<code>digest</code>	Hash di 32 byte prodotto da Argon2id

Master key e pepper risiedono entrambi in variabili d'ambiente, senza una procedura di backup né di rotazione. Il rischio è discusso nel Capitolo 12.

```
const (
    // argonTime is the number of Argon2id passes over memory.
    argonTime = 2
    // argonMemoryKiB is the working-memory size in KiB (46 MiB).
    argonMemoryKiB = 47104
    // argonThreads is the degree of parallelism.
    argonThreads = 1
    // argonKeyLen is the output hash length in bytes.
    argonKeyLen = 32
)
```

Codice 5.2: Parametri di costo di Argon2id, da `hash.go` [27].

5.2.1 Robustezza della password

Le regole di validazione della sezione 5.1 delimitano lo spazio delle password ammesse, e da quello spazio si ricava, nel caso migliore, quanta incertezza la policy possa garantire a chi tenta di indovinare una password, ossia quante possibilità debba considerare, non sapendo quale sia stata scelta.

Quanto valga quell'incertezza dipende però da come la password è stata scelta. Nel caso migliore, in cui è prodotta da un generatore pseudo-casuale che pesca dall'intero alfabeto ammesso, ogni password ammessa è equiprobabile e l'incertezza è quindi massima. Nel caso peggiore, in cui è costruita da una persona, alcune password sono ben più probabili di altre, ad esempio, `Password1!` viene scelta molto più spesso di una sequenza casuale della stessa lunghezza. Il valore calcolato qui è quello del caso migliore, cioè un limite superiore.

In questo caso, quindi, il calcolo si riduce a contare i valori distinti che la policy ammette, a partire dalla cardinalità delle quattro categorie:

n_1	=	26	minuscole
n_2	=	26	maiuscole
n_3	=	10	cifre
n_4	=	33	simboli
N	=	$\sum_i n_i = 95$	caratteri disponibili in totale

Il vincolo di almeno tre categorie esclude le stringhe che ne usano al più due, quindi per contare le password valide conviene contare quelle che non lo sono: si parte da tutte le N^L stringhe possibili e si sottraggono quelle che usano una sola categoria e quelle che ne usano esattamente due.

Il primo gruppo è immediato, perché le stringhe scritte con la sola categoria i sono n_i^L . Per il secondo non basta sommare $(n_i + n_j)^L$ su tutte le coppie, perché quel valore conta le stringhe scritte con quei due soli alfabeti e comprende quindi anche quelle di sola i e di sola j : sottraendo n_i^L e n_j^L restano le stringhe che usano davvero entrambe le categorie. Per il principio di inclusione-esclusione, il numero di password valide di lunghezza L vale quindi

$$V(L) = N^L - \sum_{i=1}^4 n_i^L - \sum_{i < j} \left[(n_i + n_j)^L - n_i^L - n_j^L \right].$$

Un attaccante privo di qualsiasi informazione deve considerare tutti i $V(L)$ candidati ammessi. Quel numero si esprime abitualmente in bit, cioè come esponente di 2: b bit corrispondono a 2^b candidati. Se la password è estratta uniformemente, l'incertezza dell'attaccante vale perciò $\log_2 V(L)$ bit.

Per la lunghezza minima ammessa, $L = 8$, si ottiene $V(8) = 6,27 \cdot 10^{15}$, cioè $\log_2 V(8) \approx 52,48$ bit.

Il confronto con lo spazio non vincolato mostra quanto pesi ciascuna delle due regole. Senza il vincolo sulle categorie sarebbero ammesse tutte le stringhe di otto caratteri sull'alfabeto, cioè $N^8 = 95^8 = 6,63 \cdot 10^{15}$, cioè 52,56 bit. La suddetta regola quindi riduce i bit di 0,08. Aggiungere un carattere in più, invece, aumenta i bit di entropia di $\log_2 95 \approx 6,57$ bit per ogni carattere aggiunto.

Da queste cifre potrebbe sembrare che il vincolo semplifichi il lavoro di un malintenzionato invece di contrastarlo, ed è vero, per quanto in misura trascurabile, nel caso in cui le password siano create da generatori pseudo-casuali. Lo scopo di quella regola però non è aumentare l'entropia, bensì vincolare la scelta di una persona, che tende a fermarsi sulle sole minuscole o su lettere e cifre. Quanto valga in quel ruolo non è ricavabile da questo calcolo.

Se le due regole debbano coesistere, o se convenga sostituire il vincolo sulle categorie con una lunghezza minima maggiore, resta quindi una questione aperta, ripresa tra gli sviluppi futuri nel Capitolo 12.

Quanto quello spazio resista dipende però anche dal costo di ogni tentativo, cioè dai parametri di Argon2id: il Capitolo 11 quantifica quel costo dal punto di vista di chi attacca.

5.3 Registrazione

Quando la richiesta raggiunge Database-Vault ed è stata rivalidata, il servizio calcola le tre trasformazioni della sezione precedente e prova a scrivere il record in una singola transazione atomica [28]. Due vincoli

di unicità proteggono la tabella, uno sull'hash dell'email e uno sulla chiave pubblica SSH: se uno dei due viene violato, la registrazione è rifiutata con HTTP 409, senza dire quale dei due abbia causato il conflitto. Quel codice resta però un'informazione: dice a un mittente non autenticato che uno dei due valori inviati era già presente, e chi invia una chiave SSH appena generata sa che l'unico altro candidato è l'email. Il Capitolo 11 analizza questo problema e presenta una possibile mitigazione.

A questo punto il record esiste, ma l'utente non ha ancora uno spazio in cui scrivere. Database-Vault genera allora un nome utente POSIX nella forma `user<xxxxxx>`, dove le sei cifre sono caratteri casuali di un alfabeto base-36, e chiede a Storage-Service di creare l'account corrispondente. Come Storage-Service lo crei e lo isoli è oggetto del Capitolo 6.

Se quella creazione fallisce, il record appena scritto viene eliminato. Se anche il rollback fallisce, il servizio lo segnala con un errore dedicato.

Quando invece entrambi i passi riescono, Database-Vault comunica a Security-Switch che la registrazione è andata a buon fine, e quest'ultimo procede con l'orchestrazione descritta nella sezione conclusiva di questo capitolo.

```
rollbackCtx, cancel := context.WithTimeout(context.WithoutCancel(ctx), rollbackTimeout)
defer cancel()

if delErr := store.DeleteUser(rollbackCtx, input.EmailHash); delErr != nil {
    return Result{
        Outcome: OutcomeFailed,
        Err: fmt.Errorf("%w: posix creation error: %w; delete error: %w", ErrOrphanedRecord,
            err, delErr),
    }
}
```

Codice 5.3: Rollback compensativo dopo un fallimento di Storage-Service, da `registration.go` [29].

La Figura 5.1 riassume l'intera sequenza, dalla richiesta del client fino alla risposta. Gli ultimi due scambi, quelli verso Network-Manager, sono oggetto della sezione conclusiva di questo capitolo.

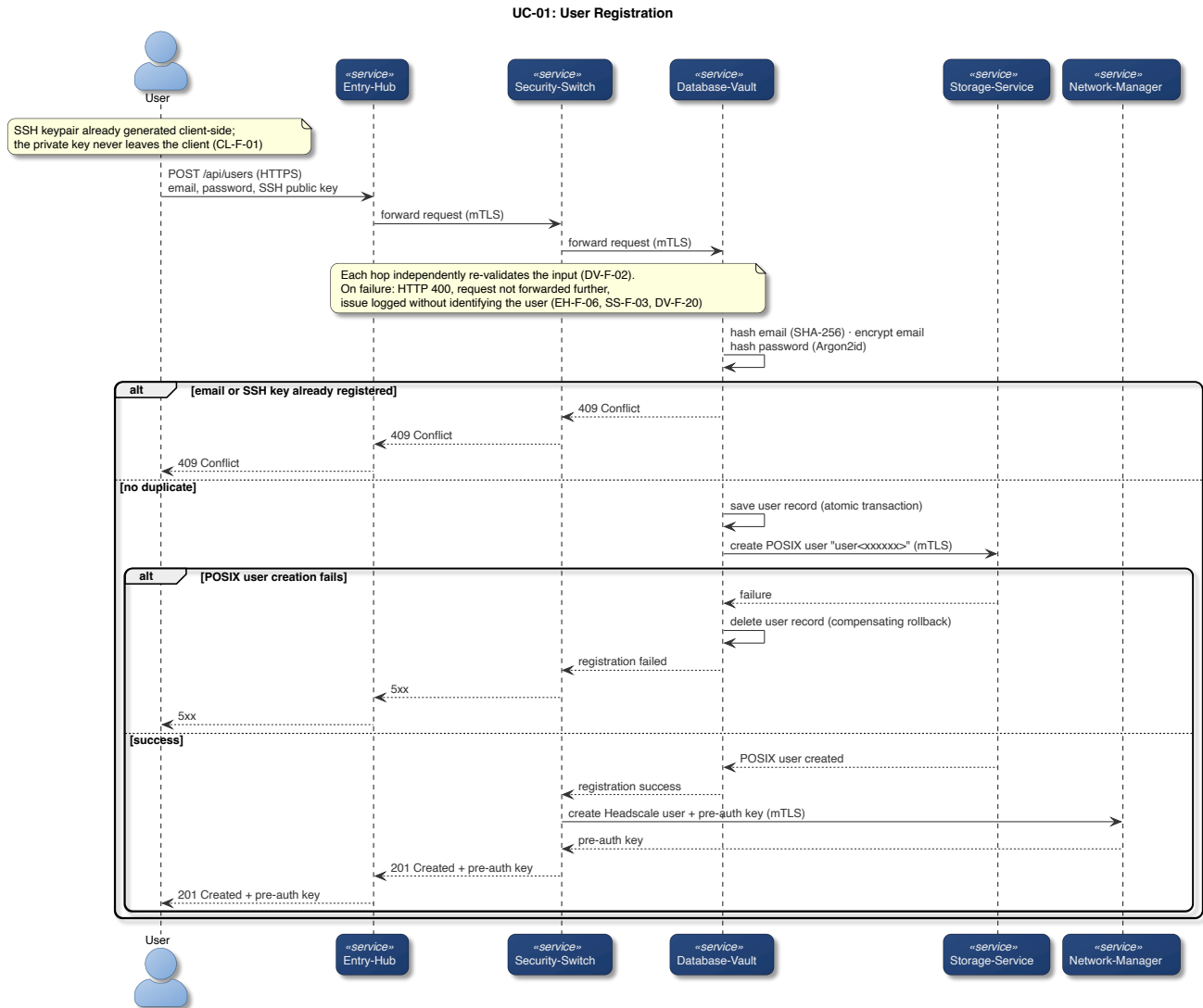


Figura 5.1: Sequenza di registrazione (UC-01).

5.4 Login

Durante il login, Database-Vault ricalcola il digest SHA-256 dell'email ricevuta e con quello cerca il record.

Salt e parametri di costo vengono estratti dalla stringa PHC del record. Argon2id viene ricalcolato con quei valori e con il pepper, e il digest ottenuto è confrontato con quello salvato tramite un confronto a tempo costante, che non lascia quindi dedurre quanti byte iniziali dei due valori coincidano.

Un'email inesistente e una password sbagliata devono produrre la stessa risposta, e non devono essere distinguibili nemmeno nei log. Le due strade convergono quindi su un unico errore sentinella, e il contenuto dell'errore originale viene scartato invece che riportato [30].

Rendere identici la risposta e il log non basta però a rendere indistinguibili i due casi, perché resta una terza differenza osservabile: il tempo. Argon2id, con i parametri della sezione precedente, costa decine di millisecondi, e se venisse eseguito solo quando un record esiste davvero, un attaccante potrebbe

distinguere le due situazioni misurando la latenza della risposta. La soluzione adottata è spendere lo stesso tempo anche quando non c'è nulla da verificare.

```
func VerifyDummy(password, pepper []byte) {
    _, _ = VerifyPassword(password, pepper, dummyHash())
}
```

Codice 5.4: Verifica fittizia usata sui percorsi di rifiuto, da `hash.go` [31].

La funzione ricalcola Argon2id contro un hash fittizio, costruito una sola volta per processo con gli stessi parametri di costo dei record veri, e ne scarta il risultato. Viene chiamata su ogni percorso di rifiuto, compreso quello in cui è il lookup stesso a fallire, così nemmeno un guasto dell'infrastruttura diventa facilmente misurabile dall'esterno.

Ogni incertezza porta allo stesso esito. Se la verifica non può concludersi, per esempio perché l'hash salvato è malformato, la risposta resta 401, secondo il principio fail-secure [15]. Nei log quel caso resta distinguibile, perché a un operatore serve sapere che si tratta di un record corrotto, di un utente distratto o di un vero e proprio attacco.

La Figura 5.2 riassume la sequenza completa di login, compreso il ramo in cui le due cause di rifiuto convergono sulla stessa risposta.

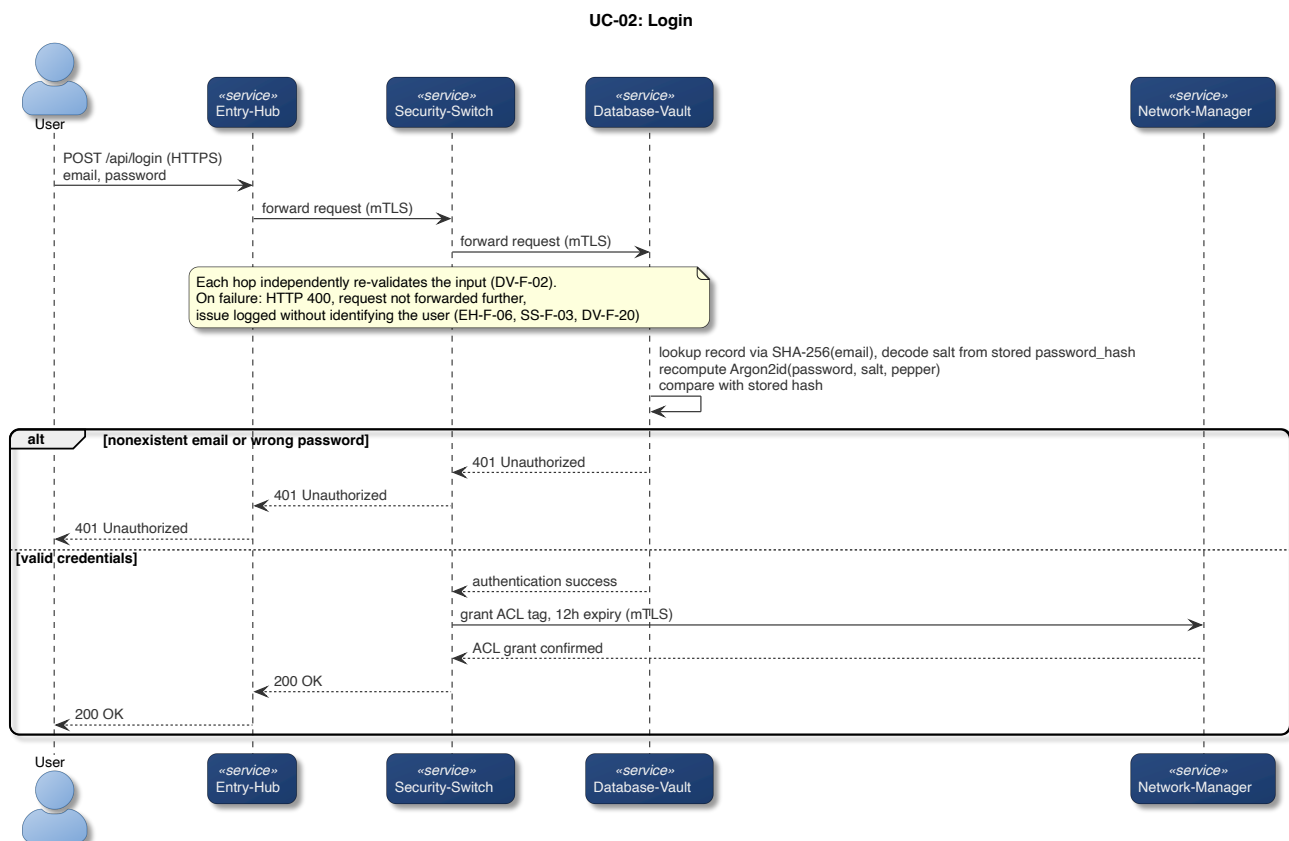


Figura 5.2: Sequenza di autenticazione (UC-02).

5.5 Orchestrazione post-autenticazione

Autenticare un utente non gli dà comunque ancora accesso a niente. L'accesso, in questo sistema, è una proprietà della rete. Security-Switch richiede a Network-Manager la modifica delle regole di rete in due momenti diversi.

Dopo una registrazione riuscita chiede la creazione dell'identità dell'utente sulla rete mesh e di una pre-auth key, che risale la catena fino al client dentro la risposta 201 e che permette al suo dispositivo di entrare nella rete privata.

Dopo un login riuscito chiede invece un accesso temporaneo di 12 ore verso Storage-Service.

Entrambi i passaggi concedono però raggiungibilità, non lettura: la registrazione mette il dispositivo nella rete, il login gli permette di raggiungere Storage-Service, ma ad aprire i backup è la chiave privata SSH. Conoscere le credenziali dell'utente non è pertanto condizione sufficiente ad accedere ai suoi backup.

Come Network-Manager crei l'identità mesh, applichi il tag ACL e ne gestisca la scadenza è oggetto del Capitolo 7.

6

Storage e backup dei file

- 6.1 Modello di accesso SFTP-only
- 6.2 Isolamento e chroot dinamico
- 6.3 Backup e restore

7

Rete e coordinamento della mesh

- 7.1 Politiche di rete e confini di reachability
- 7.2 Ciclo di vita dei grant di accesso
- 7.3 Il paradosso del coordinatore: Headscale come servizio a sé
- 7.4 CG-NAT e la necessità di un endpoint pubblico dedicato

8

Osservabilità e metriche

- 8.1 Pubblicazione delle metriche
- 8.2 Ingestione e controllo di coerenza
- 8.3 Persistenza delle serie temporali
- 8.4 Visualizzazione delle metriche

9

Testing e validazione

- 9.1 Il modello V e i quattro livelli di test
- 9.2 Strategia di integrazione bottom-up
- 9.3 TDD: criteri di ingresso e uscita
- 9.4 Copertura e lacune del piano di test
- 9.5 Risultati: verifiche completate

10

Deployment

- 10.1 Containerizzazione e supervisione s6-overlay
- 10.2 Perché non tutti i container sono distroless
- 10.3 Docker locale vs Proxmox VE: due livelli di isolamento
- 10.4 Topologia distribuita
- 10.5 Alta disponibilità e resilienza: stato attuale e limiti

Analisi critica delle proprietà di sicurezza

11.1 Scenari di attacco e mitigazioni

11.2 Costo di un attacco alle password

Il tempo necessario a esaurire lo spazio è il rapporto tra il numero di candidati e la velocità con cui l'attaccante li verifica:

$$T = \frac{V(L)}{R},$$

dove R è il numero di tentativi al secondo che il suo hardware sostiene. Il numeratore dipende da come la password è stata scelta, il denominatore da come viene verificata, ed è su quest'ultimo che si concentra la strategia difensiva del sistema.

Essendo Argon2id memory-hard, ogni tentativo deve muovere attraverso la memoria almeno due volte i propri 46 MiB, quindi la banda dell'acceleratore fissa un tetto massimo al numero di tentativi al secondo, indipendentemente da quanto sia efficiente l'implementazione.

Applichiamo quel tetto alla più grande infrastruttura documentata. Il cluster Stargate di Abilene contava 201.600 acceleratori Blackwell operativi al 18 agosto 2026 [32], e NVIDIA dichiara 576 TB/s di banda per ogni rack da 72 GPU, cioè 8 TB/s ciascuna [33]. Dividendo la banda complessiva per il traffico di un tentativo si ottengono al più $1,7 \cdot 10^{10}$ tentativi al secondo.

A quel ritmo i $6,27 \cdot 10^{15}$ candidati di una password di otto caratteri si esaurirebbero in poco più di quattro giorni, in media due. Il tetto considera però la sola banda, e ignora sia il costo aritmetico della funzione di mescolamento sia la latenza degli accessi dipendenti dai dati, che nelle implementazioni reali mordono prima: il valore ottenuto è quindi un limite superiore alla velocità dell'attaccante, e i quattro giorni un limite inferiore al tempo che gli servirebbe.

Il conto presuppone inoltre che l'attaccante disponga del pepper oltre che del database, cioè che abbia compromesso il processo di Database-Vault e non soltanto i suoi dati: senza quel valore nessun tentativo è calcolabile. Il salt, che invece è nel record e quindi in suo possesso, non rallenta la singola ricerca ma la costringe a essere mirata, perché nessun tentativo vale per due utenti insieme: i quattro giorni sono il costo per una password, non per l'intero archivio.

Il confronto con una password più lunga mostra dove stia davvero la difesa. Con dodici caratteri sullo stesso alfabeto i candidati diventano $5,4 \cdot 10^{23}$, e lo stesso cluster impiegherebbe circa un milione di anni: quattro caratteri in più spostano il risultato di otto ordini di grandezza, mentre raddoppiare i passaggi di Argon2id si limiterebbe a raddoppiare il tempo.

11.3 Isolamento dei fallimenti

11.4 Costo delle scelte architetturali

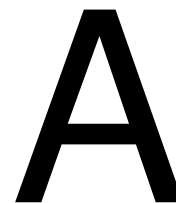
12

Conclusioni

12.1 Risultati raggiunti

12.2 Limiti noti

12.3 Sviluppi futuri



Elenco completo dei requisiti

Le tabelle seguenti raccolgono i requisiti di sistema funzionali, non funzionali e di dominio, organizzati secondo la stessa struttura dell'SRS originale. [1]

Il capitolo 3 discute per intero i requisiti utente, i quali sono gli unici a non essere inclusi nelle tabelle sottostanti.

A.1 Requisiti di sistema funzionali

A.1.1 Client

Tabella A.1: Requisiti funzionali del Client.

ID	Requisito
CL-F-01	Deve generare autonomamente una coppia di chiavi SSH (pubblica/privata) prima della registrazione, senza mai trasmettere la chiave privata ad alcun componente del sistema.
CL-F-02	Su comando dell'utente, deve inviare a Entry-Hub, durante la registrazione (<code>POST /api/users</code>), solo la chiave pubblica SSH, insieme a email e password.
CL-F-03	Su comando dell'utente, deve ripetere il login (<code>POST /api/login</code>) prima della scadenza del grant ACL di 12 ore, per mantenere la continuità di accesso a Storage-Service.
CL-F-04	Deve configurare e avviare Tailscale usando la pre-auth key ricevuta nella risposta di registrazione, per unirsi alla rete mesh privata.
CL-F-05	Deve risolvere Storage-Service tramite MagicDNS sulla rete mesh, senza fare affidamento su indirizzi IP statici.
CL-F-06	Su comando dell'utente, deve invocare <code>restic backup</code> verso Storage-Service via SFTP, autenticandosi con la chiave privata SSH generata in CL-F-01.
CL-F-07	Su comando dell'utente, deve invocare <code>restic restore</code> verso Storage-Service via SFTP, usando lo stesso metodo di autenticazione di CL-F-06.

ID	Requisito
CL-F-08	Deve gestire i codici di errore HTTP restituiti da Entry-Hub (400/401/403/500/502/503/504) senza esporre dettagli interni del sistema all'utente finale.
CL-F-09	Su comando dell'utente, deve validare localmente email, password e — solo in fase di registrazione — la chiave pubblica SSH, usando le stesse regole imposte da Entry-Hub (EH-F-04 per la registrazione, EH-F-05 per il login), prima di inviare <code>POST /api/users</code> o <code>POST /api/login</code> ; in caso di fallimento della validazione locale, non deve trasmettere la richiesta.

A.1.2 Entry-Hub

Tabella A.2: Requisiti funzionali di Entry-Hub.

ID	Requisito
EH-F-01	Deve esporre un endpoint HTTPS pubblico di health-check (<code>GET /api/health</code>), con certificati firmati dalla CA pubblica Let's Encrypt.
EH-F-02	Deve esporre un endpoint HTTPS pubblico per la registrazione degli utenti (<code>POST /api/users</code>), con certificati firmati dalla CA pubblica Let's Encrypt.
EH-F-03	Deve esporre un endpoint HTTPS (<code>POST /api/login</code>) per l'autenticazione degli utenti registrati, servito su un listener separato raggiungibile solo dalla rete mesh privata (non da internet, a differenza del listener di EH-F-01/EH-F-02), con lo stesso certificato firmato dalla CA pubblica Let's Encrypt presentato da EH-F-01/EH-F-02 — non mTLS; cambia solo la raggiungibilità di rete.
EH-F-04	<code>/api/users</code> deve accettare JSON e validare: presenza dei campi email, password e chiave pubblica SSH; dimensione del payload entro un limite definito; assenza di campi aggiuntivi inattesi; formato dell'email (RFC 5322); lunghezza della password tra 8 e 128 caratteri; complessità della password (almeno 3 categorie di caratteri tra minuscole, maiuscole, cifre, simboli); correttezza formale della chiave pubblica SSH.
EH-F-05	<code>/api/login</code> deve accettare JSON e validare: presenza dei campi email e password; dimensione del payload entro un limite definito; assenza di campi aggiuntivi inattesi; formato dell'email (RFC 5322); lunghezza della password tra 8 e 128 caratteri; complessità della password (almeno 3 categorie di caratteri).
EH-F-06	In caso di fallimento della validazione deve: rispondere con HTTP 400 (Bad Request) senza specificare quale problema sia stato riscontrato; registrare nel log il problema trovato senza identificare l'utente; non inoltrare la richiesta ad alcun altro servizio interno.

ID	Requisito
EH-F-07	In caso di validazione riuscita deve: registrare nel log l'esito della validazione senza identificare l'utente; inoltrare la richiesta a Security-Switch via mTLS; verificare che il certificato provenga da un Security-Switch; verificare la validità del certificato X.509.
EH-F-08	Deve inoltrare all'utente la risposta ricevuta da Security-Switch.
EH-F-09	Deve mappare gli errori interni sui codici HTTP 400/401/500/502/503, restituendo al client messaggi sanificati e mantenendo i log dettagliati solo internamente.
EH-F-10	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (<code>metrics/Entry-Hub</code>), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
EH-F-11	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.

A.1.3 Security-Switch

Tabella A.3: Requisiti funzionali di Security-Switch.

ID	Requisito
SS-F-01	Deve accettare solo connessioni mTLS da client che soddisfino: campo <code>organization</code> uguale a <code>EntryHub</code> ; certificato X.509 valido; accesso alla rete mesh privata.
SS-F-02	Deve ri-validare l'input ricevuto, indipendentemente dalla validazione già effettuata da Entry-Hub.
SS-F-03	In caso di fallimento della validazione deve: rispondere con HTTP 400 (Bad Request) senza specificare quale problema sia stato riscontrato; registrare nel log il problema trovato senza identificare l'utente; non inoltrare la richiesta ad alcun altro servizio interno.
SS-F-04	In caso di validazione riuscita deve: registrare nel log l'esito della validazione senza identificare l'utente; inoltrare la richiesta a Database-Vault via mTLS, verificando che: il certificato provenga da un Database-Vault; il certificato X.509 sia valido.
SS-F-05	Dopo la conferma di autenticazione riuscita da Database-Vault, deve richiedere a Network-Manager (via mTLS) di concedere a quell'utente l'accesso a Storage-Service per 12 ore.
SS-F-06	Deve mappare gli errori sui codici HTTP 400/401/403/500/502/504.
SS-F-07	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (<code>metrics/Security-Switch</code>), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
SS-F-08	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.

ID	Requisito
SS-F-09	Dopo la conferma di registrazione riuscita da Database-Vault, deve richiedere a Network-Manager (via mTLS) di creare un utente Headscale dedicato e generare una pre-auth key per il nuovo account, includendo poi quella chiave nella risposta a Entry-Hub.

A.1.4 Database-Vault

Tabella A.4: Requisiti funzionali di Database-Vault.

ID	Requisito
DV-F-01	Deve accettare solo connessioni mTLS da client che soddisfino: campo <code>organization</code> uguale a <code>SecuritySwitch</code> ; certificato valido; accesso alla rete mesh privata.
DV-F-02	Deve ri-validare l'input ricevuto, indipendentemente dalla validazione già effettuata da Security-Switch.
DV-F-03	Deve calcolare l'hash SHA-256 dell'email per l'indicizzazione e come chiave primaria, senza mai registrare nel log l'email in chiaro.
DV-F-04	Deve cifrare l'email dell'utente: derivare una chiave di cifratura per record dalla master key con HKDF-SHA256 e un salt casuale a 16 byte, poi cifrare l'email con AES-256-GCM usando quella chiave derivata e un nonce casuale a 12 byte.
DV-F-05	La master key dovrebbe provenire da una sorgente configurabile, con validazione della lunghezza (32 byte).
DV-F-06	Deve mantenere un pepper come variabile d'ambiente.
DV-F-07	Deve calcolare l'hash della password con Argon2id: memoria 47104 KiB (46 MiB), 2 iterazioni, parallelismo 1, output a 32 byte, usando un salt casuale per record e il pepper (DV-F-06).
DV-F-08	Deve salvare il record utente in una transazione atomica.
DV-F-09	Deve richiedere a Storage-Service la creazione dell'utente POSIX univoco sul server, con username <code>user<xxxxxx></code> (xxxxxx: 6 caratteri casuali da un alfabeto base-36), e attenderne la risposta.
DV-F-10	Se la creazione dell'utente POSIX fallisce, deve eliminare l'utente dal database e informare Security-Switch che la registrazione è fallita.
DV-F-11	Dopo aver creato il record utente e l'utente POSIX, deve informare Security-Switch che l'utente è stato registrato.
DV-F-12	Deve rifiutare (HTTP 409) le registrazioni con un'email o una chiave SSH già esistenti, senza fornire dettagli sull'errore.
DV-F-13	Durante il login, deve recuperare il salt associato all'email tramite l'hash SHA-256 dell'email (DV-F-03).
DV-F-14	Deve ricalcolare Argon2id sulla password ricevuta usando il salt recuperato e il pepper, confrontando il risultato con l'hash memorizzato.

ID	Requisito
DV-F-15	Deve rispondere con lo stesso codice HTTP 401 sia per un'email inesistente sia per una password errata, senza distinguere i due casi né nella risposta né nel log.
DV-F-16	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (<code>metrics/Database-Vault</code>), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
DV-F-17	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
DV-F-18	Dovrebbe esistere una procedura di backup della master key.
DV-F-19	Dovrebbe esistere una procedura di rotazione della master key.
DV-F-20	In caso di fallimento della validazione deve: rispondere con HTTP 400 (Bad Request) senza specificare quale problema sia stato riscontrato; registrare nel log il problema trovato senza identificare l'utente; non inoltrare la richiesta ad alcun altro servizio interno.

A.1.5 Storage-Service

Tabella A.5: Requisiti funzionali di Storage-Service.

ID	Requisito
ST-F-01	Deve accettare connessioni mTLS solo da client che soddisfino: campo <code>organization</code> uguale a <code>DatabaseVault</code> ; certificato valido; accesso alla rete mesh privata.
ST-F-02	Deve fornire upload/download di file già cifrati lato client, senza mai elaborare contenuto in chiaro.
ST-F-03	L'accesso deve avvenire esclusivamente via SFTP, autenticato con la chiave pubblica SSH registrata dall'utente.
ST-F-04	Deve vietare esplicitamente qualsiasi forma di connessione SSH diversa da SFTP.
ST-F-05	Ogni utente deve disporre di uno spazio di storage isolato, non accessibile dagli altri utenti.
ST-F-06	Su richiesta di Database-Vault via mTLS, deve creare sul sistema un utente POSIX con username <code>user<xxxxxx></code> (<code>xxxxxx</code> : 6 caratteri casuali da un alfabeto base-36).
ST-F-07	Deve garantire che l'utente POSIX non possa mai uscire dalla propria directory.
ST-F-08	L'account POSIX creato non deve avere una home directory tradizionale né una shell interattiva; l'unico spazio scrivibile è la sottodirectory dedicata all'interno del chroot.
ST-F-09	La configurazione sshd di Storage-Service deve avere <code>PasswordAuthentication no</code> e <code>PermitRootLogin no</code> , indipendentemente dal fatto che gli account creati non abbiano una password impostata.
ST-F-10	A seguito della creazione, riuscita o fallita, dell'utente POSIX, deve riportare l'esito a Database-Vault.

ID	Requisito
ST-F-11	Ad ogni tentativo di connessione SFTP dell'utente, deve recuperare la chiave pubblica corrente dell'utente da Database-Vault tramite <code>AuthorizedKeysCommand</code> .
ST-F-12	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (<code>metrics/Storage-Service</code>), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
ST-F-13	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
ST-F-14	Dovrebbe imporre limiti di quota per utente.
ST-F-15	Dovrebbe garantire: replica automatica dei dati; tolleranza ai guasti per almeno un nodo; consistenza dei dati; possibilità di espansione tramite l'aggiunta di nuovi nodi senza interrompere il servizio.

A.1.6 Network-Manager

Tabella A.6: Requisiti funzionali di Network-Manager.

ID	Requisito
NM-F-01	Deve garantire che solo Entry-Hub, Database-Vault, Network-Manager e Certificate-Authority possano contattare Security-Switch.
NM-F-02	Deve garantire che solo Security-Switch, Storage-Service e Certificate-Authority possano contattare Database-Vault.
NM-F-03	Deve garantire che solo Security-Switch e Certificate-Authority possano contattare Network-Manager.
NM-F-04	Deve garantire che tutti i componenti interni della rete, eccetto gli utenti, possano contattare ed essere contattati dalla Certificate-Authority sulla rete mesh.
NM-F-05	Deve garantire che solo gli utenti autenticati possano vedere e contattare Storage-Service.
NM-F-06	Deve garantire che gli utenti registrati ma non autenticati possano vedere e contattare solo Entry-Hub.
NM-F-07	Deve garantire che gli utenti registrati e autenticati possano vedere e contattare solo Entry-Hub e Storage-Service.
NM-F-08	Su richiesta di Security-Switch, a seguito di una registrazione riuscita, deve creare un utente Headscale dedicato e generare una pre-auth key a breve durata per il nuovo account.
NM-F-09	Dopo un login riuscito, su richiesta di Security-Switch, deve assegnare al nodo dell'utente il tag ACL che abilita la raggiungibilità verso Storage-Service, registrando una scadenza a 12 ore da quel momento.
NM-F-10	Deve verificare periodicamente le scadenze registrate e rimuovere il tag ACL dai nodi scaduti, automaticamente e senza intervento manuale.

ID	Requisito
NM-F-11	La scadenza di ogni grant deve essere persistita, non mantenuta solo in memoria, per non perdere lo stato in caso di riavvio di Network-Manager.
NM-F-12	La creazione di pre-auth key e la gestione dei tag ACL devono essere ristrette specificamente a Network-Manager, verificato via mutual TLS (campo <code>organization</code> uguale a <code>NetworkManager</code>).
NM-F-13	La pre-auth key serve unicamente a registrare il nodo come membro della mesh; da sola, non concede raggiungibilità verso Storage-Service.
NM-F-14	L'endpoint di coordinamento di Headscale è deliberatamente raggiungibile dalla rete pubblica (idealmente ospitato su un proprio VPS con indirizzo pubblico).
NM-F-15	Deve configurare MagicDNS con un dominio di base dedicato, in modo che Storage-Service sia risolvibile da tutti i nodi della mesh tramite un nome stabile anziché un IP.
NM-F-16	Il nodo mesh di Network-Manager deve accettare la configurazione DNS distribuita da Headscale (MagicDNS), in modo che le proprie chiamate in uscita verso Certificate-Authority e MQTT-Broker risolvano quegli hostname sulla mesh, anziché ricadere su un percorso di rete privo di equivalente in produzione.
NM-F-17	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (<code>metrics/Network-Manager</code>), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
NM-F-18	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.

A.1.7 Certificate-Authority

Tabella A.7: Requisiti funzionali di Certificate-Authority.

ID	Requisito
CA-F-01	Deve garantire che i componenti che presentano certificati per mTLS siano realmente chi dichiarano di essere.
CA-F-02	Deve garantire l'emissione, la rotazione, la revoca e la verifica dei certificati mTLS.
CA-F-03	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (<code>metrics/Certificate-Authority</code>), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
CA-F-04	Deve accettare un bootstrap token monouso, distribuito fuori banda a ciascun servizio, per l'emissione del certificato iniziale; i rinnovi successivi devono avvenire tramite il certificato mTLS corrente, non tramite il token.

A.1.8 Sistema di monitoraggio

Tabella A.8: Requisiti funzionali del sistema di monitoraggio (MQTT-Broker / Metrics-Collector / TimescaleDB / Grafana).

ID	Requisito
MT-F-01	Metrics-Collector può leggere solo <code>metrics/*</code> .
MT-F-02	Metrics-Collector deve scartare le metriche il cui campo <code>service</code> non corrisponde al topic da cui provengono.
MT-F-03	Le metriche devono essere memorizzate come hypertable TimescaleDB, con retention automatica di 30 giorni e compressione dopo 7 giorni.
MT-F-04	Devono esistere dashboard Grafana per tempo di risposta, throughput e connessioni attive.

A.1.9 Infrastruttura di rete e PKI

Tabella A.9: Requisiti funzionali di rete e PKI.

ID	Requisito
PKI-F-01	Ogni servizio deve autenticarsi mutuamente con certificati X.509 emessi da una CA valida.
PKI-F-02	Ogni servizio deve verificare il campo <code>organization</code> del certificato, non solo la sua validità.
PKI-F-03	Dovrebbe esistere una procedura di rotazione e revoca dei certificati.
NET-F-01	La comunicazione tra servizi deve avvenire sulla rete privata; gli endpoint pubblici di Entry-Hub e l'endpoint di coordinamento Headscale di Network-Manager (NM-F-14) sono le uniche superfici deliberatamente esposte pubblicamente dal sistema.
NET-F-02	Il TLS utilizzato deve essere in versione 1.3.

A.2 Requisiti non funzionali

A.2.1 Di prodotto

Tabella A.10: Requisiti non funzionali di prodotto.

ID	Requisito
RNF-SEC-01	Zero-knowledge: nessun componente server-side accede mai al contenuto dei file di backup in chiaro, poiché la cifratura avviene lato client prima della trasmissione. Questo non si estende alle credenziali di accesso: email e password transitano, cifrate (TLS/mTLS), attraverso Entry-Hub, Security-Switch e Database-Vault per la validazione e l'hashing, pur senza mai essere persistite o registrate nei log in chiaro (cfr. DV-F-03, RD-01).

ID	Requisito
RNF-SEC-02	Zero-trust: nessun servizio deve fidarsi implicitamente dei dati ricevuti da un altro, anche se autenticato via mTLS.
RNF-SEC-03	Defense-in-depth: ogni livello ri-valida l'input in modo indipendente.
RNF-SEC-04	Tutta la comunicazione tra servizi deve usare mTLS, senza eccezioni.
RNF-REL-01	Il sistema deve tollerare la compromissione isolata di un singolo componente, senza che questa si propaghi agli altri.
RNF-PERF-01	La latenza delle richieste HTTP deve essere tracciata (p50/p95/p99).
RNF-USA-01	I messaggi di errore devono essere comprensibili e correttamente categorizzati per codice HTTP.
RNF-MAINT-01	Ogni servizio deve poter essere isolato, ri-certificato e riavviato individualmente senza impattare gli altri.

A.2.2 Organizzativi

Tabella A.11: Requisiti non funzionali organizzativi.

ID	Requisito
RNF-ORG-01	Linguaggio di implementazione: Go.
RNF-ORG-03	Licenza open-source MIT.
RNF-ORG-04	Piattaforma di deployment: Proxmox VE (KVM per Storage-Service, Database-Vault, Network-Manager; LXC per gli altri servizi).
RNF-ORG-05	Sviluppo ed esercizio garantiti su macOS e Linux (con Docker).

A.2.3 Esterni

Tabella A.12: Requisiti non funzionali esterni.

ID	Requisito
RNF-EXT-01	Poiché il sistema elabora dati personali (l'email), dovrebbe rispettare le normative sulla privacy applicabili (es. GDPR). Attualmente fuori perimetro.

A.3 Requisiti di dominio

Tabella A.13: Requisiti di dominio (RD).

ID	Requisito
RD-01	Qualsiasi nuovo componente introdotto in futuro non deve creare un percorso lungo il quale dati sensibili in chiaro attraversino o vengano registrati da un componente diverso dal client o dal componente strettamente necessario alla loro cifratura/decifratura.
RD-02	Le chiavi derivate (tramite HKDF) non devono mai essere persistite: qualsiasi nuovo requisito di memorizzazione di chiavi deve essere valutato rispetto a questo vincolo prima di essere accettato.
RD-03	Argon2id e AES-256-GCM sono vincoli tecnologici non negoziabili.
RD-04	Il principio di <i>fail-secure</i> si applica a ogni componente: in caso di incertezza sulla validità di una richiesta, il comportamento predefinito è negare l'accesso.

Bibliografia

- [1] Francesco Verrengia. RAM-USB software requirements specification. https://github.com/Verryx-02/RAM-USB/blob/main/docs/Software_Requirements_Specification.md, 2026.
- [2] Barry W. Boehm. A spiral model of software development and enhancement. *IEEE Computer*, 21(5):61–72, 1988. <https://doi.org/10.1109/2.59>.
- [3] Francesco Verrengia. RAM-USB software requirements specification, v1.0: original wording of NM-F-12. https://github.com/Verryx-02/RAM-USB/blob/3ef723e07a0292337fc48a1cd2e8c952f2f2e2ee/docs/Software_Requirements_Specification.md?plain=1#L260, 2026.
- [4] Francesco Verrengia. RAM-USB: correct NM-F-12 wording and document how headscale’s CLI satisfies it. <https://github.com/Verryx-02/RAM-USB/commit/975be1f467873628532a807e852f086dba29a9ef>, 2026.
- [5] Francesco Verrengia. RAM-USB: rework NM-F-12/NM-F-14/NET-F-01 for a standalone public headscale and retire EH-F-12. <https://github.com/Verryx-02/RAM-USB/commit/f2d17f41668ae2c978c849ac308ac4fa52f7b409>, 2026.
- [6] Francesco Verrengia. RAM-USB contributing guidelines: Commit structure (section 2.2). <https://github.com/Verryx-02/RAM-USB/blob/7984a140e0ce7f6b3ea89470801c0d26ecbd4aa1/CONTRIBUTING.md?plain=1#L33-L101>, 2026.
- [7] Google. distroless: Language focused docker images, minus the operating system. <https://github.com/GoogleContainerTools/distroless>, 2026.
- [8] Murugiah Souppaya, John Morello, e Karen Scarfone. Application container security guide. <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-190.pdf>, 2017. NIST Special Publication 800-190.
- [9] Michael Wager. Evaluating the security of containers through reduction of potentially vulnerable components. https://www.mwager.de/assets/component_reduction_paper.pdf, 2024. Hochschule Augsburg.
- [10] Josh Aas, Richard Barnes, Benton Case, Zakir Durumeric, Peter Eckersley, Alan Flores-López, J. Alex Halderman, Jacob Hoffman-Andrews, James Kasten, Eric Rescorla, Seth Schoen, e Brad Warren. Let’s Encrypt: An automated certificate authority to encrypt the entire web. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS ’19)*, pp. 2473–2487. ACM, 2019. <https://doi.org/10.1145/3319535.3363192>.

- [11] Francesco Verrengia. RAM-USB: decodeJSON and ValidateRegister. <https://github.com/Verryx-02/RAM-USB/blob/7c1e991f953a4c686e09f8142d0b3c1d9d64fc5f/pkg/validation/validation.go#L125-L173>, 2026.
- [12] Scott Rose, Oliver Borchert, Stu Mitchell, e Sean Connelly. NIST special publication 800-207: Zero Trust Architecture. <https://doi.org/10.6028/NIST.SP.800-207>, 2020.
- [13] Francesco Verrengia. RAM-USB: shared internal libraries. <https://github.com/Verryx-02/RAM-USB/tree/main/pkg>, 2026.
- [14] restic. restic: Fast, secure, efficient backup program. <https://github.com/restic/restic>, 2026.
- [15] OWASP. Fail securely. https://owasp.org/www-community/Fail_securely, 2026.
- [16] Smallstep. certificates: A private certificate authority (x.509 & ssh) & acme server for secure automated certificate management. <https://github.com/smallstep/certificates>, 2026.
- [17] Juan Font. headscale: An open source, self-hosted implementation of the tailscale control server. <https://github.com/juanfont/headscale>, 2026.
- [18] Timescale. TimescaleDB: A time-series database for high-performance real-time analytics. <https://github.com/timescale/timescaledb>, 2026.
- [19] Grafana Labs. Grafana: The open and composable observability and data visualization platform. <https://github.com/grafana/grafana>, 2026.
- [20] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Tesi di Dottorato di Ricerca, University of California, Irvine, 2000. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [21] Peter W. Resnick. Internet message format. <https://doi.org/10.17487/RFC5322>, 2008. RFC 5322.
- [22] Hugo Krawczyk e Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). <https://doi.org/10.17487/RFC5869>, 2010. RFC 5869.
- [23] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. <https://doi.org/10.6028/NIST.SP.800-38D>, 2007. NIST Special Publication 800-38D.
- [24] Francesco Verrengia. RAM-USB: derivazione della chiave per record con HKDF-SHA256. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/encryption/encryption.go#L88-L98>, 2026.
- [25] Alex Biryukov, Daniel Dinu, Dmitry Khovratovich, e Simon Josefsson. Argon2 memory-hard function for password hashing and proof-of-work applications. <https://doi.org/10.17487/RFC9106>, 2021. RFC 9106.

- [26] OWASP. Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2026.
- [27] Francesco Verrengia. RAM-USB: parametri di costo Argon2id di database-vault. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/password/hash.go#L33-L42>, 2026.
- [28] Francesco Verrengia. RAM-USB: salvataggio del record utente in transazione atomica. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/storage/storage.go#L111-L146>, 2026.
- [29] Francesco Verrengia. RAM-USB: rollback compensativo dopo un fallimento di storage-service. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/registration/registration.go#L157-L165>, 2026.
- [30] Francesco Verrengia. RAM-USB: esito unico del login per email inesistente e password errata. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/login/login.go#L160-L185>, 2026.
- [31] Francesco Verrengia. RAM-USB: verifica fittizia che pareggia i tempi di risposta del login. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/password/hash.go#L178-L193>, 2026.
- [32] Epoch AI. OpenAI Stargate Abilene: AI data center directory. <https://epoch.ai/data/ai-data-centers/directory/openai-stargate-abilene>, 2026. Dati al 18 agosto 2026.
- [33] NVIDIA. NVIDIA GB300 NVL72: specifiche tecniche. <https://www.nvidia.com/en-gb/data-center/gb300-nvl72>, 2026.